

PCS-302 : Data Structures Lab

Two Stacks Using One Array | Space-Efficient Stack Sharing

Problem Statement

Design an algorithm and a program to implement two stacks by using only one array, i.e., both stacks should use the same array for push and pop operations. The array should be divided in such a manner that space is used efficiently if one stack contains a very large number of elements and the other one contains only a few elements.

Example

Array of size 6, two stacks growing toward each other:

Index: 0 1 2 3 4 5

```
push1(10)  10
push1(20)  10  20
push2(100)                                100
push2(200)                                200 100
```

Stack1 (left to right): 10 20 (top1 = 1)

Stack2 (right to left): 200 100 (top2 = 4)

Stacks meet when $\text{top1} + 1 == \text{top2}$ --> Overflow

Algorithm

```
Create():
    top1 = -1                // Stack 1 grows from the left
    top2 = N                 // Stack 2 grows from the right
```

```
Push1(k):
    if top1 + 1 == top2: print "Overflow", return
    top1 = top1 + 1
    arr[top1] = k
```

```
Push2(k):
    if top1 + 1 == top2: print "Overflow", return
    top2 = top2 - 1
    arr[top2] = k
```

```
Pop1():
    if top1 == -1: print "Underflow", return
    remove arr[top1]
    top1 = top1 - 1
```

```
Pop2():
    if top2 == N: print "Underflow", return
    remove arr[top2]
    top2 = top2 + 1
```

This is efficient because both stacks share the SAME array and

grow toward each other -- one stack can use almost the whole array as long as the other stays small, unlike splitting the array into two fixed halves in advance.

C Program

```
#include <stdio.h>
#define N 6

int arr[N];
int top1 = -1;    // Stack 1 grows from left (index 0 upward)
int top2 = N;     // Stack 2 grows from right (index N-1 downward)

void push1(int k) {
    if (top1 + 1 == top2) {
        printf("Overflow! Cannot push %d to Stack1.\n", k);
        return;
    }
    top1 = top1 + 1;
    arr[top1] = k;
    printf("%d pushed to Stack1.\n", k);
}

void push2(int k) {
    if (top1 + 1 == top2) {
        printf("Overflow! Cannot push %d to Stack2.\n", k);
        return;
    }
    top2 = top2 - 1;
    arr[top2] = k;
    printf("%d pushed to Stack2.\n", k);
}

void pop1() {
    if (top1 == -1) {
        printf("Stack1 Underflow.\n");
        return;
    }
    printf("%d popped from Stack1.\n", arr[top1]);
    top1 = top1 - 1;
}

void pop2() {
    if (top2 == N) {
        printf("Stack2 Underflow.\n");
        return;
    }
    printf("%d popped from Stack2.\n", arr[top2]);
    top2 = top2 + 1;
}

void printStacks() {
    int i;
    printf("Stack1: ");
    for (i = 0; i <= top1; i++)
```

```

        printf("%d ", arr[i]);
    printf("\n");

    printf("Stack2: ");
    for (i = N - 1; i >= top2; i--)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    push1(10);
    push1(20);
    push2(100);
    push2(200);

    printStacks();

    pop1();
    pop2();

    printStacks();

    return 0;
}

```